# INHERITANCE

# Chapter 6 Objectives

After completing this module, the student will be able to:

- Implement multiple classes with the same interface
- Understand the concept of polymorphism
- Use polymorphism in real programs
- Use virtual functions to define desired interfaces to be fulfilled by objects
- Use objects through interfaces instead of concrete implementations
- Implement interfaces according to specifications
- Compose behaviour of several objects into a single interface implementation
- Model real-world entities in C++
- Use objects and classes in C++ in a more advanced way
- Verify user input
- Implement useful and informative data presentation techniques

# 6.1 DEFINING CLASS HIERARCHY

# Inheritance

- Inheritance is an “is-a” relationship
  - Example: “every employee is a person”
- Inheritance lets us create new classes from existing classes
  - New classes are called the derived classes
  - Existing classes are called the base classes
- Derived classes inherit the properties of the base classes

# Examples

| Base Class | Derived Classes |
|---|---|
| Student | Graduate Student, Undergraduate Student |
| Shape | Circle, Rectangle |
| Employee | Faculty Member, Staff Member |

# Inheritance (continued)

- Inheritance can be viewed as a tree-like, or hierarchical, structure wherein a base class is shown with its derived classes

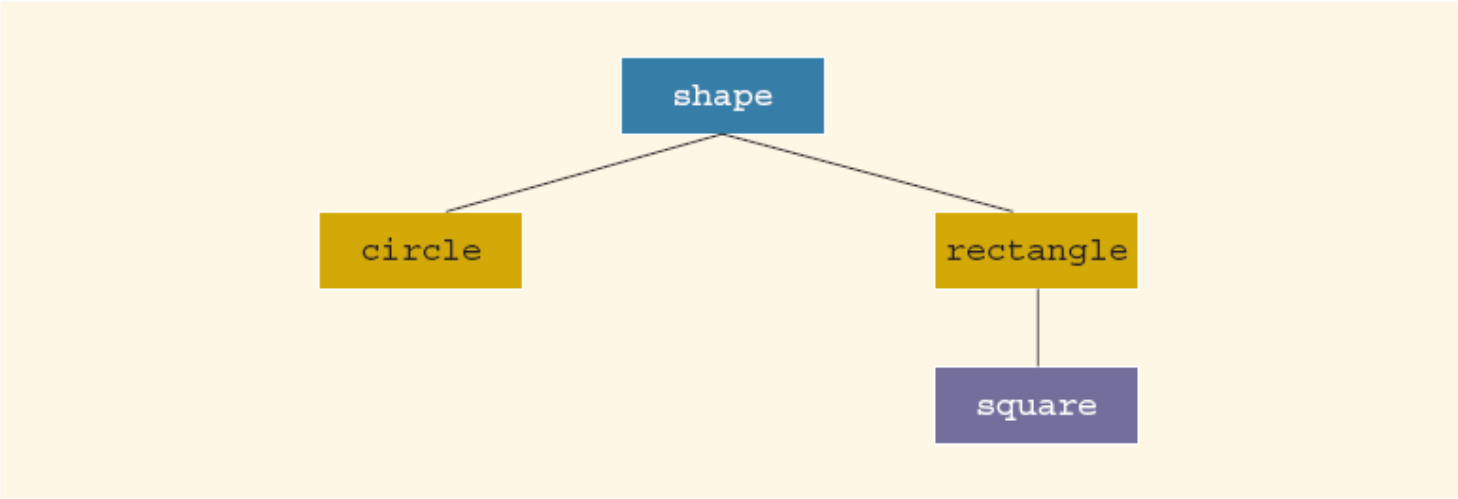



FIGURE 13-1 Inheritance hierarchy

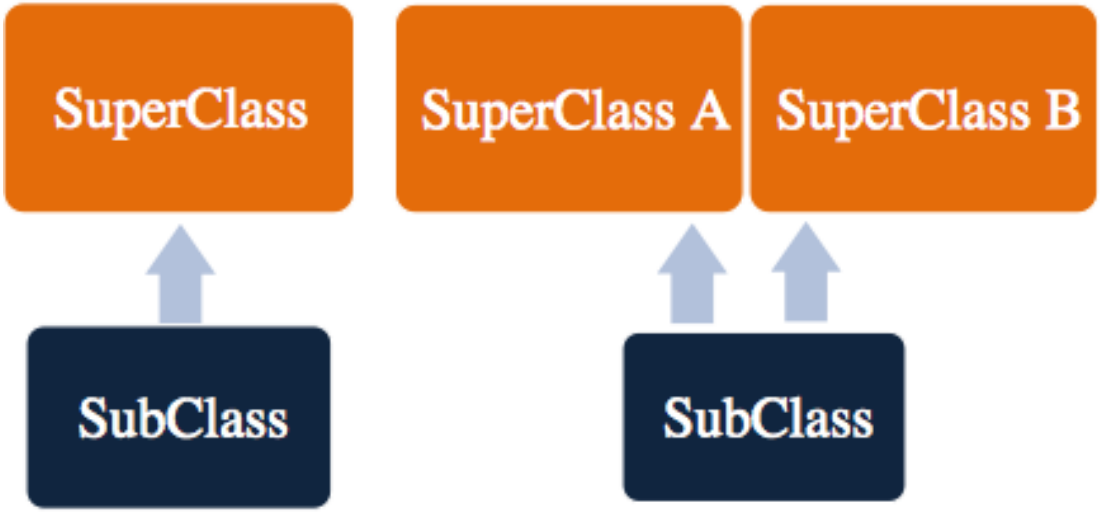
# Types of Inheritance

- **Based on the number of base classes**
  - **Single inheritance:** Derived class has one base class
    - Example: Circle derived from Shape
  - **Multiple inheritance:** Derived class has more than one base class
    - Example: Son derived from Mother and Father

- **Based on the access level of inherited members**
  - **Public inheritance:** Public members of the base class remain public in the derived class
  - **Protected inheritance:** Public and protected members of the base class become protected
  - **Private inheritance:** Public and protected members of the base class become private

# Types of Inheritance

- The left diagram illustrates a "**single inheritance**", and the right one a "**multiple inheritance**" or "**multi-inheritance**".
- A class that is **inherited from** can be called a **base class, superclass,** or **parent class**, while the class that **inherits** is called a **derived class, subclass,** or **child class**.



# Inheritance (continued)

- General syntax of a derived class:

```
Class Subclass: MemberAccessSpecifier Superclass
{           };
```

- Where the Member Access Specifier is public, protected, or private (default)
- The private members of a base class are private to the derived class
- Derived class cannot directly access them
- Example.

```
Class Circle: public Shape
{           };
```

# Defining a simple subclass (3)

- If we want to **define a class named Y** as a **subclass o**f a **superclass named X**, we use the following syntax

```
Class Y: MemberAccessSpecifier X
{           };
```

- If there's **more than one superclass**, we have to enlist them all using **commas as separators**, like this:

```
Class Y: MemberAccessSpecifier X, Z
{           };
```

# Defining a simple subclass (2)

The class on the right → will serve as **a superclass**. Analyse its structure, it's not difficult, we promise.

The program emits the following text:

**101**

```
#include <iostream>
using namespace std;
class Super {
private:
	int storage;
public:
	void put(int val) { storage = val; }
	int get(void) { return storage; }
};
int main(void) {
	Super object;

	object.put(100);
	object.put(object.get() + 1);
	cout << object.get() << endl;
	return 0;
}
```

# Defining a simple subclass (4)

We have defined a class named *Sub*, which is a subclass of a class named *Super*. We may also say that the *Sub* class is derived from the *Super* class.

The *Sub* class introduces neither new variables nor new functions. Does this mean that any object of the *Sub*class inherits all the traits of the *Super* class, being in fact a copy of the *Super* class's objects?

No. It doesn't.

```
class Sub : Super {
};

int main(void) {
    Sub object;

    object.put(100);
    object.put(object.get() + 1);
    cout << object.get() << endl;
    return 0;
}
```

# Defining a simple subclass (4) cont'd

- If we compile the previous code, we will get compilation errors saying that the **put** and **get methods** are inaccessible. Why?

- When we omit the visibility specifier, private inheritance is assumed by default. This means:
  - All **public members** of the superclass become **private** in the subclass.
  - All **private members** of the superclass remain **inaccessible**.
  - As a result, you cannot use these members directly inside the subclass.

# Defining a simple subclass (4) cont'd

- We have to tell the compiler that **we want to preserve the previously used access policy**. We do this by using a "public" visibility specifier:

```
Class Subclass: public Superclass
{          };
```

- Don't be misled: this doesn't mean that the private components of the *Super*class (like the ***storage*** variable) will magically turn into public ones. Private components will remain private, public components will remain public.

# Defining a simple subclass (5)

- Objects of the *Sub* class may do almost the same things as their older siblings created from the *Super*class.
- We use the word 'almost' because being a subclass also means that **the class has lost access to the private components of the superclass**.
- We cannot write a member function of the *Sub* class that would be able to manipulate the storage variable directly.

```
#include <iostream>

using namespace std;

class Super {
private:
    int storage;
public:
    void put(int val) { storage = val; }
    int get(void) { return storage; }
};

class Sub : public Super {
};

int main(void) {
    Sub object;

    object.put(100);
    object.put(object.get() + 1);
    cout << object.get() << endl;
    return 0;
}
```

# Defining a simple subclass (5) cont'd

This is a very serious restriction. Is there any workaround?

Yes.

There's the third access level we haven't mentioned yet. It's called "**protected**".

The keyword *protected* means that any component marked with it **behaves like a public component when used by any of the subclasses and looks like a private component to the rest of the world**.

We should add that this is true only for publicly inherited classes (like the *Super* class in our example previous example)

Let's make use of the keyword right now.

# Defining a simple subclass (6)

- In the example code, we have added some new functionality to the *Sub* class.
- We have added the ***print*** function to access the ***storage*** variable from the Superclass. This wouldn't be possible if the variable were declared as **private**.
- In the ***main*** function scope, the variable remains hidden anyway. You cannot write anything like this:

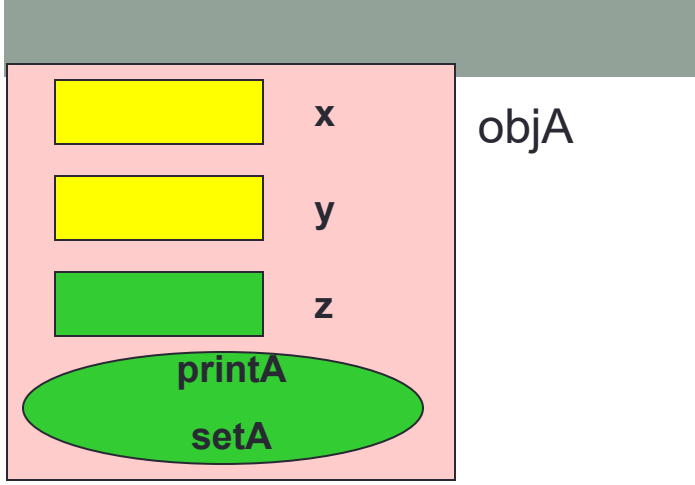
  object.storage = 0;
- The program will produce the following output:
- **storage = 101**

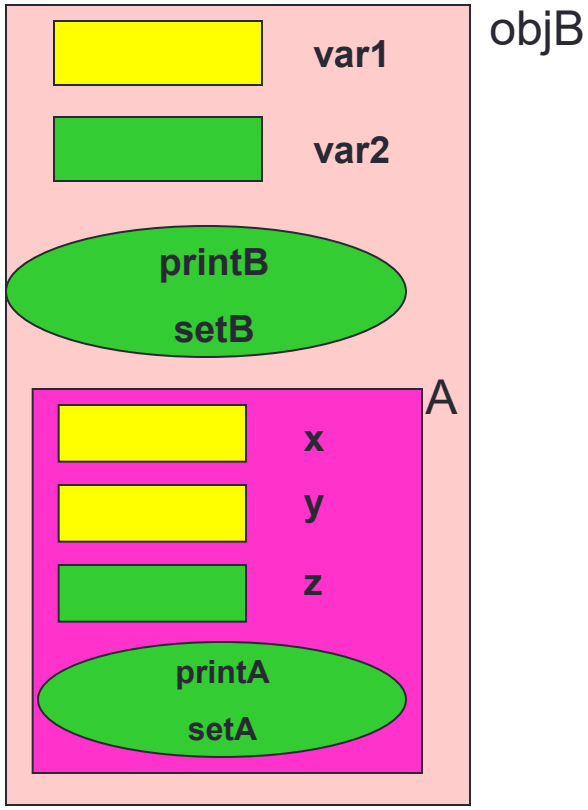
```
#include <iostream>

using namespace std;

class Super {
protected:
    int storage;
public:
    void put(int val) { storage = val; }
    int get(void) { return storage; }
};

class Sub : public Super {
public:
    void print(void) { cout << "storage = " << storage << endl; }
};

int main(void) {
    Sub object;

    object.put(100);
    object.put(object.get() + 1);
    object.print();
    return 0;
}
```

# Protected Members of a Class

- Private members of a class cannot be directly accessed outside the class
- For a base class to give derived class access to a `private` member
  - Declare that member as `protected`
- The accessibility of a `protected` member of a class is in between `public` and `private`
  - A derived class can directly access the `protected` member of the base class

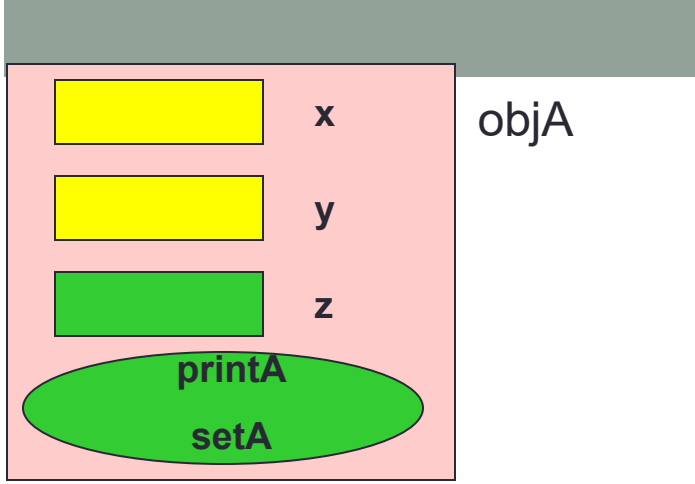
```
#include <iostream>
using namespace std;;

class A                Base
{
    int x;
    int y;
public:
    int z;
    void printA(){cout<<x<<"  "<<y<<"   "<<endl;}
    void setA(int a, int b, int c){x=a; y=b; z=c;}
};

class B: public A   derived
{
    int var1;
public:
    int var2;
    void setB(int a, int b) {var1=a; var2=b;}
    void printB(){cout<<var1<<"  "<<var2<<endl;}
};

void main()
{
    A objA;
    B objB;
}
```

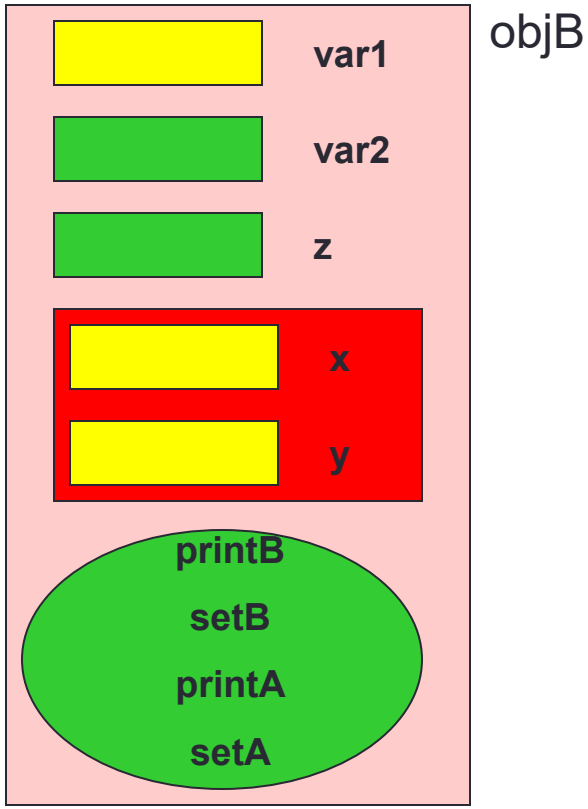


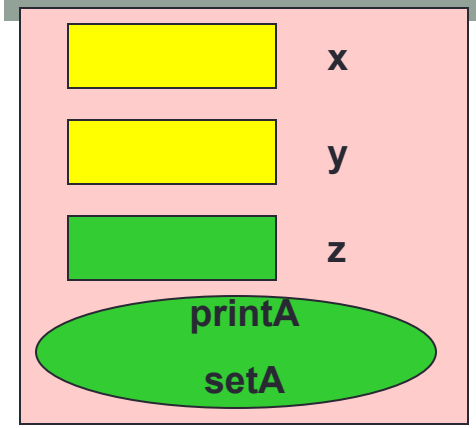
```
#include <iostream>
using namespace std;;

class A
{
    int x;
    int y;
public:
    int z;
    void printA(){cout<<x<<"  "<<y<<"   "<<endl;}
    void setA(int a, int b, int c){x=a; y=b; z=c;}
};

class B: public A
{
    int var1;
public:
    int var2;
    void setB(int a, int b) {var1=a; var2=b;}
    void printB(){cout<<var1<<"  "<<var2<<endl;}
};

void main()
{
    A objA;
    B objB;
}
```

Can not access x, y

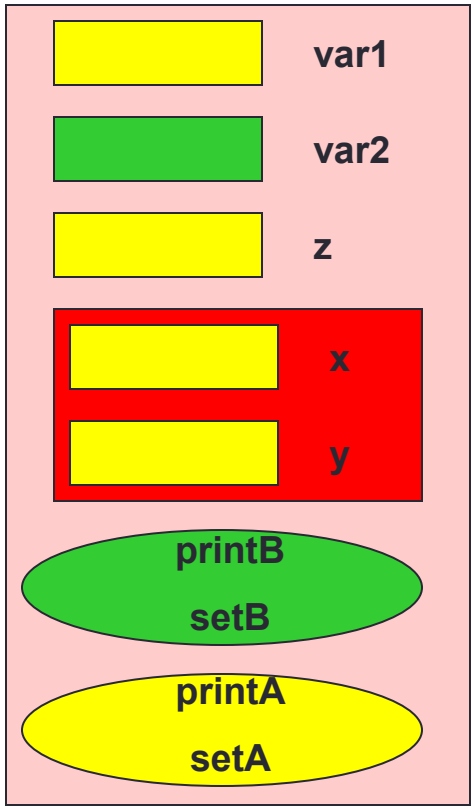


```
#include <iostream>
using namespace std;;

class A
{
    int x;
    int y;
public:
    int z;
    void printA(){cout<<x<<"  "<<y<<"   "<<endl;}
    void setA(int a, int b, int c){x=a; y=b; z=c;}
};

class B: private A
{
    int var1;
public:
    int var2;
    void setB(int a, int b) {var1=a; var2=b;}
    void printB(){cout<<var1<<"  "<<var2<<endl;}
};

void main()
{
    A objA;
    B objB;
}
```

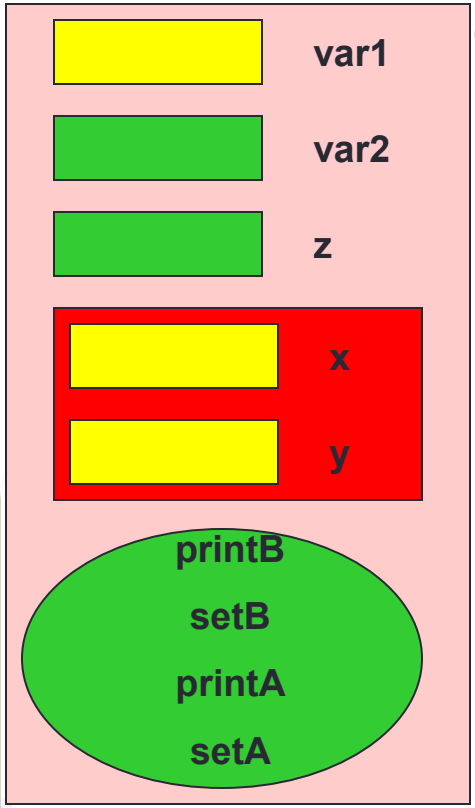



objA




objB

```
#include <iostream>
using namespace std;;
class A
{
    int x;
    int y;
public:
    int z;
    void printA(){cout<<x<<"   "<<y<<"   "<<z<<endl;}
    void setA(int a, int b, int c){x=a; y=b; z=c;}
};
class B:public A
{
    int var1;
public:
    int var2;
    void setB(int a, int b) {var1=a; var2=b;}
    void printB(){cout<<var1<<"   "<<var2<<endl;}
};

void main()
{
    B objB;
    objB.setA(3,4,5);
    objB.setB(7,8);
    objB.printA();
    objB.printB();
}
```



# Defining a simple subclass (7)

Now's a good opportunity to do a little summarising here. The component of the class may be declared as:

- public
- private
- Protected

These three keywords may also be used in a completely different context to specify the visibility inheritance model. So far, we've talked about public and private keywords used in such a case. The protected keyword can also be employed in this role.

# Defining a simple subclass (7) cont'd

- The table shows the possible combinations of component declarations and inheritance types, along with the resulting access to the components once the subclass is fully defined.

- Familiarize yourself with this table, which is a basic tool for resolving all issues related to the inheritance of class members.

| When the component is declared as: | When the class is inherited as: | The resulting access inside the subclass is: |
|---|---|---|
| public | public | Public |
| protected | | protected |
| private | | none |
| public | protected | protected |
| protected | | protected |
| private | | none |
| public | private | private |
| protected | | private |
| private | | none |

# Inheritance as public, protected, or private

Class **B**: MemberAccessSpecifier **A** { };

- If memberAccessSpecifier is public:
  - public members of A are public members of B and can be directly accessed in class B
  - protected members of A are protected members of B and can be directly accessed by member functions (and friend functions) of B
  - private members of A are hidden in B and can be accessed by member functions of B through public or protected members of A

# Inheritance as public, protected, or private

Class **B**: MemberAccessSpecifier **A** { };

- If memberAccessSpecifier is protected:
  - public members of A are protected members of B and can be accessed by the member functions (and friend functions) of B
  - protected members of A are protected members of B and can be accessed by the member functions (and friend functions) of B
  - private members of A are hidden in B and can be accessed by member functions of B through public or protected members of A

# Inheritance as public, protected, or private

Class **B**: MemberAccessSpecifier **A** { };

- If memberAccessSpecifier is private:
  - public members of A are private members of B and can be accessed by member functions of B
  - protected members of A are private members of B and can be accessed by member functions (and friend functions) of B
  - private members of A are hidden in B and can be accessed by member functions of B through public/protected members of A

```
class A
{
public:
    int x;
private:
    int y;
protected:
    int z;
};

class B:public A
{
public:
    void fun()
    {
        x=3;
    //  y=4; illegal
        z=5;
    }
protected:
    int w;
};


void main()
{
B objB;
objB.fun();
//objB.w=7;   illegal
objB.x=5;
//objB.y=12;  illegal
//objB.z=10;  illegal
}
```

```
class A
{
public:
    int x;
private:
    int y;
protected:
    int z;
};

class B:protected A
{
public:
    void fun()
    {
        x=3;
    //  y=4; illegal
        z=5;
    }
protected:
    int w;
};


void main()
{
B objB;
objB.fun();
//objB.w=7;   illegal
//objB.x=5;   illegal
//objB.y=12;  illegal
//objB.z=10;  illegal
}
```

```
class A
{
public:
    int x;
private:
    int y;
protected:
    int z;
};

class B:private A
{
public:
    void fun()
    {
        x=3;
    //  y=4; illegal
        z=5;
    }
protected:
    int w;
};


void main()
{
B objB;
objB.fun();
//objB.w=7;   illegal
//objB.x=5;
//objB.y=12;  illegal
//objB.z=10;  illegal
}
```

# Redefining (Overriding) Member Functions of the Base Class

- To redefine (override) a `public` member function of a base class
  - Corresponding function in the derived class must have the same name, number, and types of parameters. Redefined
  - If the function has a different signature, this will become function overloading.

# Redefining (Overriding) Member Functions of the Base Class

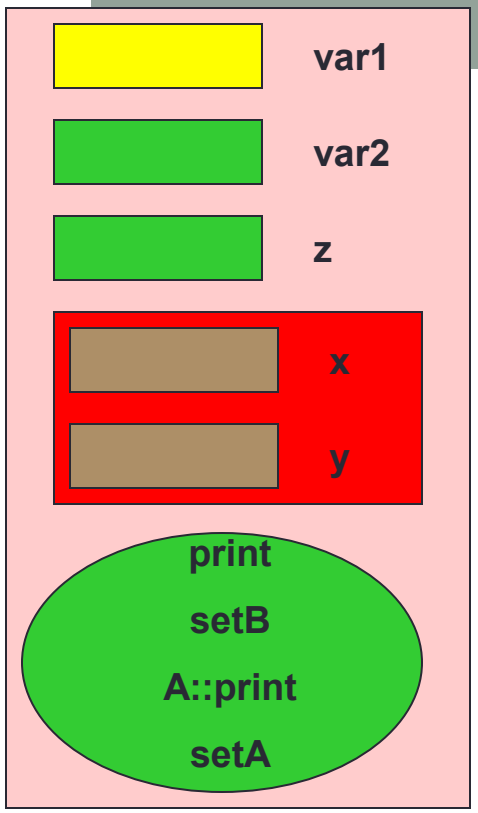
- `public` members of the base class can be inherited as `public` or `private` members
- The derived class can include additional members, i.e., data and/or functions
- The derived class can redefine the `public` member functions of the base class
- All members of the base class are also member variables of the derived class

# Redefining (Overriding) Member Functions of the Base Class (continued)

If derived class overrides a `public` member function of the base `class`, then to call the base class function, specify:

- Name of the base class
- Scope resolution operator (`::`)
- Function name with the appropriate parameter list

```
#include <iostream>
using namespace std;;
class A
{
    int x;
    int y;
public:
    int z;
    void setA(int a, int b, int c){x=a; y=b; z=c;}
    void print(){cout<<x<<"  "<<y<<"   "<<z<<endl;}
};
class B:public A
{
    int var1;
public:
    int var2;
    void setB(int a, int b) {var1=a; var2=b; }
    void setB(int a) {var1=a; var2=a;}
    void print(){cout<<var1<<"  "<<var2<<endl;  A::print(); }
};

void main()
{
    B objB;
    objB.setB(7,8);
    objB.setA(1,2,3);
    objB.print();
    objB.A::print();
}
```

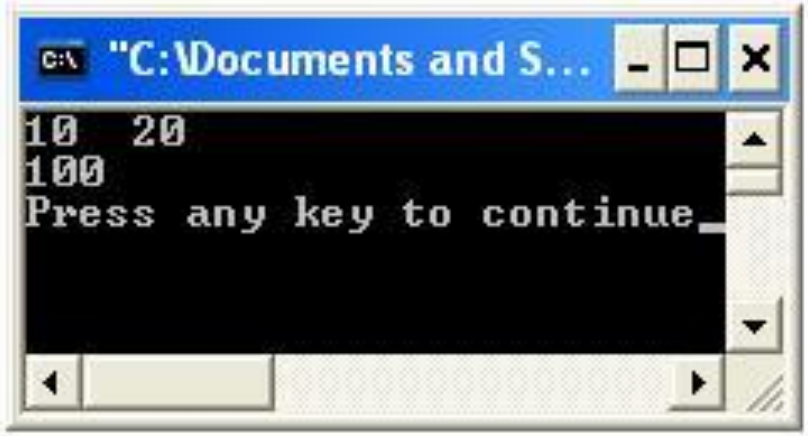


# Constructors of Derived and Base Classes

- Derived class constructor cannot directly access `private` members of the base class
- Derived class can directly initialize only `public` member variables of the base class
- When a derived object is declared
  - It must execute one of the base class constructors
- Call to base class constructor is specified in heading of derived class constructor definition

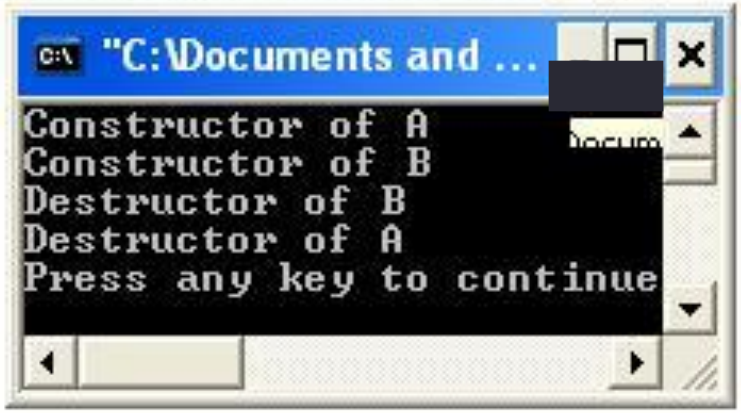
# Example

```
class A
{
public:
    int x;
    void print()
    {cout<<x<<"   "<<y<<endl;}
    A() { x=10; y=20;}
private:
    int y;
};

class B:public A
{
public:
    int z;
    void print() {A::print(); cout<<z<<endl;}
    B(){ z=100;}
};

void main()
{
    B objB;
    objB.print();
}
```



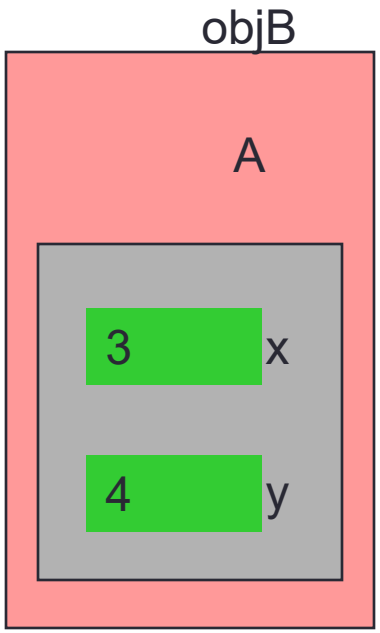
# Example

```
class A
{
public:
    int x;
    A() { x=10; y=20; cout<< "Constructor of A"<<endl;}
    ~A(){cout<<"Destructor of A"<<endl;}
private:
    int y;
};

class B:public A
{
public:
    int z;
    B(){z=100; cout<<"Constructor of B"<<endl;}
    ~B(){cout<<"Destructor of B"<<endl;}
};

void main()
{
    B objB;
}
```



## Initialization List and Inheritance

```
class A
{
public:
    int x;
    A(int a,int b) { x=a; y=b;}
private:
    int y;
};

class B:public A
{
public:
    B(int a,int b): A(a,b){}
};

void main()
{
    B objB(3,4);
}
```




Initialization list goes with constructor of derived class and uses class name to pass parameters to base class constructor.

# Summary

- Inheritance and composition are meaningful ways to relate two or more classes
- Inheritance is an “is-a” relation
  - Single inheritance: a derived class is derived from one class, called the base class
  - Multiple inheritance: a derived class is derived from more than one base class

# Summary (continued)

- Private members of a base class are private to the base class
- Public members of a base class can be inherited either as public or private
- Derived class can redefine function members of a base class
  - Redefinition applies only to objects of derived class

# Summary (continued)

- A call to a base class constructor (with parameters) is specified in the heading of the definition of the derived class constructor
- When initializing object of a derived class, the base class constructor is executed first
- In composition
  - Class member is an object of another class
  - Call to constructor of member objects is specified in heading of the definition of class's constructor

# Summary (continued)

- Three basic principles of OOD are:
  - Encapsulation
  - Inheritance
  - Polymorphism

- Finding classes:
  - Describe the problem
  - Choose classes from the list of nouns
  - Choose operations from the list of verbs